01 · CONTEXTO

RED SOCIAL Y ALERTA DE MASCOTAS PERDIDAS

RimayPet



Cuando una mascota se pierde, la comunidad responde

RimayPet une a los dueños de mascotas en una comunidad digital y activa una alerta geolocalizada para encontrar a quien se perdió.

01 · CONTEXTO

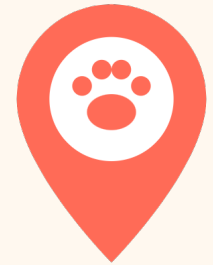
Ayacucho ama a sus mascotas, pero no tiene dónde avisar cuando una se pierde

64,0 %

de los hogares del Perú tiene al menos una mascota.

47,8 %

de los hogares ayacuchanos tiene gatos: una de las tres regiones con mayor proporción.



Hoy, quien pierde una mascota publica en grupos genéricos o recurre a canales concentrados en la capital. **RimayPet es el espacio propio que faltaba.**

Alerta cercana
Avisa a quien vive cerca

Comunidad local
Hecha por y para Ayacucho

Costo cero
Infraestructura gratuita

Fuente: INEI (2025), Tenencia y crianza de mascotas 2025.

01 · CONTEXTO

Qué es RimayPet y qué ofrece

Comunidad

Perfiles de mascotas, feed de publicaciones y mensajes directos en tiempo real.

Alerta

Reporta una mascota perdida y avisa a quienes están dentro del radio que configures. El estado se comparte con un enlace público.

Confianza

Perfiles verificados con carnet de vacunación, privacidad configurable y moderación.

Una sola plataforma para compartir el día a día y actuar rápido en la emergencia. Pensada para:

Dueños de mascotas | **Veterinarias aliadas** | **Refugios de adopción**

Análisis: lo que condiciona el diseño

Requisitos

15 requisitos funcionales y 25 historias de usuario en seis épicas: cuenta, comunidad, mensajería, mascotas perdidas, descubrimiento y moderación.

Calidad medible

6 escenarios con fuente, estímulo, respuesta y medida: rendimiento, disponibilidad, escalabilidad, seguridad, mantenibilidad y observabilidad.

Drivers

Mandan dos: alertas masivas sin bloquear y escalar cada función por separado.

Restricciones: infraestructura gratuita, un solo desarrollador, REST con WebSocket, Caddy + API con JWT y contenedores en más de un servidor.

Cómo se valida: con pruebas de carga de 1000 usuarios y pruebas de acceso con y sin JWT.

Fuente: Bass et al. (2021).

02 · ANÁLISIS

Historias de usuario y requisitos

Cuenta y perfiles

5 historias · RF01, RF06, RF09

Registrarse, crear perfiles y fijar mi radio de alerta

Publicaciones y comunidad

5 historias · RF02, RF03, RF10, RF14

Publicar, seguir, comentar y crear grupos

Mensajería y notificaciones

3 historias · RF04, RF11

Mensajes en tiempo real y avisos push

Mascotas perdidas

5 historias · RF05, RF06

Reportar, avistar, actualizar y compartir el estado

Descubrimiento

2 historias · RF12, RF13

Buscar y recibir sugerencias por raza y cercanía

Moderación y auditoría

5 historias · RF07, RF08, RF15

Reportar, revisar, suspender y auditar

25 historias de usuario en seis épicas, priorizadas con MoSCoW: 11 Debe, 10 Debería y 4 Podría; cubiertas por **15 requisitos funcionales**.

Requisitos no funcionales y restricciones

Requisitos no funcionales

Rendimiento feed y mensajería con baja latencia

Disponibilidad operativa durante una alerta

Escalabilidad 1000 usuarios, escalando cada servicio

Seguridad HTTPS, JWT, contraseñas con Argon2 y bitácora

Mantenibilidad nuevos módulos sin afectar a los existentes

Restricciones

RC01 infraestructura gratuita en Oracle Free Tier

RC02 un solo desarrollador

RC03 REST y un canal WebSocket aparte

RC04 acceso por Caddy + API con JWT

RC05 contenedores Docker en más de un servidor

02 · ANÁLISIS

Calidad medible y con qué la medimos

≤ 2 s

Rendimiento

95 % de consultas del feed con 1000 usuarios

k6 (pruebas de carga)

≤ 1 s

Disponibilidad

Respuesta al reportar con el envío masivo en curso

k6 (carga con alerta activa)

1000

Escalabilidad

Usuarios, con el 95 % de respuestas en 3 s

k6 y docker stats

100 %

Seguridad

Intentos sin JWT válido rechazados

Postman y Newman

0

Mantenibilidad

Archivos de dominio cambiados al cambiar de proveedor

node --test en GitHub Actions

≤ 5 min

Observabilidad

Para ubicar el paso fallido de una alerta

Registros con Pino

Medidas propuestas, por validar con pruebas de carga (escenarios EQ-01 a EQ-06).

03 · ARQUITECTURA

Seis módulos, una sola comunidad

Perfiles y moderación
Cuentas, mascotas, privacidad, verificación y reportes

Publicaciones y feed
Fotos, videos, reacciones y comentarios

Mensajería
Conversaciones en tiempo real por WebSocket

Alertas
Reportes, avistamientos y estado público

Notificaciones
Avisos y alertas push a la comunidad

Identidad
Registro, autenticación, roles y tokens

Drivers y decisiones arquitectónicas

Driver	Decisión	ADR
DA01 Concurrencia de 1000	API sin estado + caché + réplica condicional	ADR-001, 009, 010
DA02 Alerta sin bloquear	Transactional Outbox + worker	ADR-003
DA03 Proteger los datos	Caddy HTTPS + JWT + autorización	ADR-005
DA04 Dos VM y un desarrollador	Docker Compose + Oracle Free Tier	ADR-008
DA05 REST y WebSocket	PWA + API REST/WS	ADR-006
DA06 Cambiar sin romper	Clean Architecture + fachadas	ADR-002, 003

Además: PostgreSQL/PostGIS con esquemas por módulo (ADR-004), medios en R2 (ADR-007), caché del feed (ADR-009) y Outbox transaccional (ADR-003).

Fuente: registros de decisión arquitectónica (Nygard, 2011).

03 · ARQUITECTURA ★ CLAVE

Estilo: monolito modular



Una sola aplicación contiene seis módulos. Alertas guarda el reporte y el evento Outbox en una transacción; el worker procesa la notificación después.

Monolito modular

Los módulos comparten un artefacto y una base; la API puede replicarse como unidad completa si las métricas lo justifican.

Eventos

Alertas registra el evento en Outbox y responde; el worker entrega la notificación sin bloquear la petición (DA02).

Se descartan microservicios

El monolito modular reduce la sobrecarga operativa y mantiene fronteras internas verificables.

Fuente: Bass et al. (2021), Martin (2017) y ADR-001 a ADR-011.

03 · ARQUITECTURA ★ CLAVE

Flujo de ejemplo: reportar una mascota perdida



1
Dueño reporta
La PWA envía ubicación y foto

2
API valida
JWT en cada petición

3
Alertas guarda
PostGIS y respuesta inmediata

4
Outbox registra
Evento en la misma transacción

5
Worker procesa
Notificaciones envía push

Pasos 1 a 3: se confirma el reporte y su evento Outbox; la API no espera el envío push.

Pasos 4 y 5: el aviso masivo sigue aparte y no frena al feed ni a la mensajería.

Enfoque: Clean Architecture



Dominio

Reglas: reporte, ubicación, radio, estados



Aplicación

Casos de uso y puertos



Presentación

Controladores REST, WebSocket y eventos



Infraestructura

PostGIS, Outbox, CDN y servicio push



Las dependencias apuntan siempre al dominio.

Cambiar de proveedor no toca las reglas del negocio.

Aplicación completa en Alertas y Perfiles, donde están las reglas. **Más ligera** en Publicaciones, Mensajería y Worker procesa.

Fuente: Martin (2017).

Modelo C4: del contexto al código

C1
Contexto
Dueño, moderador, CDN y servicio push

C2
Contenedores
PWA, Caddy, API, worker y PostgreSQL

C3
Componentes
Servicio de Alertas por dentro

C4
Código
Caso de uso ReportarMascotaPerdida

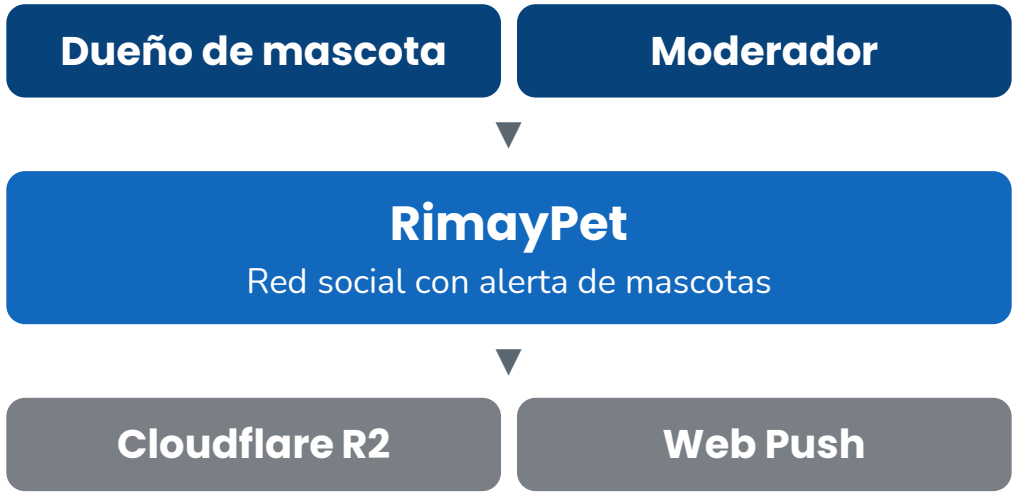
Trazabilidad: Notificaciones ← RF10/RF11 ← HU05 ← DA02 ← ADR-003.

Dónde verlo: carpeta docs/04-modelo-c4 del repositorio. Los niveles 1 y 2 sirven para explicar el sistema a cualquier persona; los niveles 3 y 4, al equipo técnico.

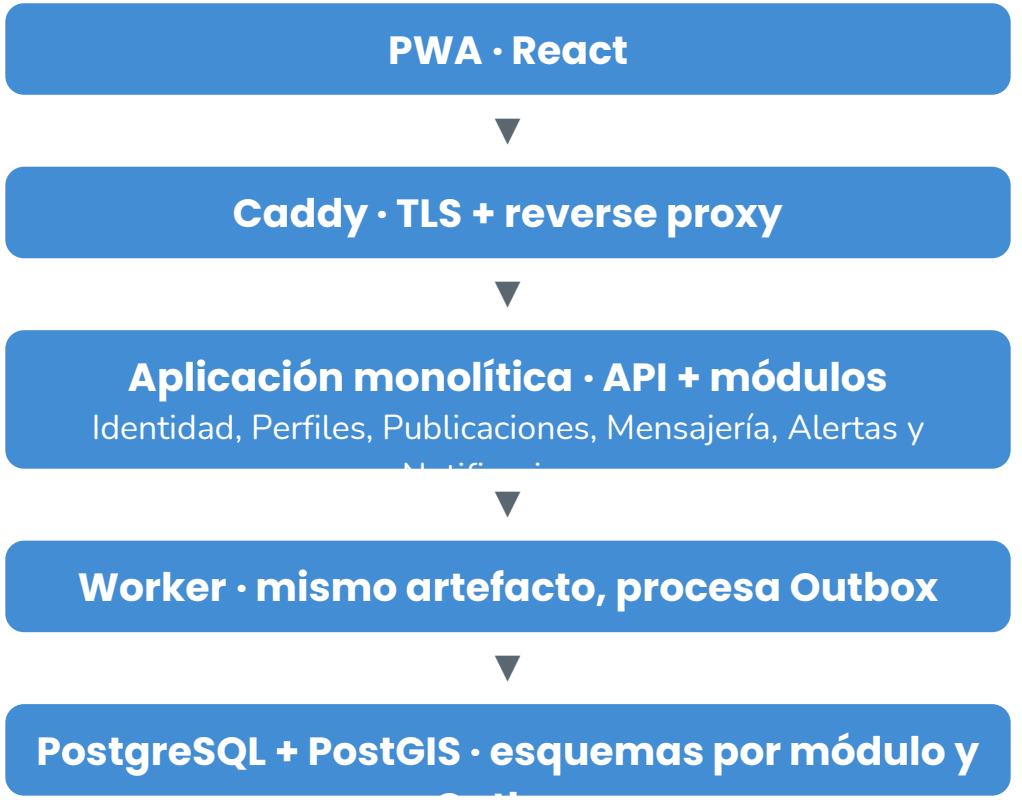
Fuente: Brown (s. f.), The C4 model.

C4 niveles 1 y 2: contexto y contenedores

C1 · Contexto



C2 · Contenedores



Fuente: Brown (s. f.), The C4 model.

C4 niveles 3 y 4: componentes y código

C3 · Módulo Alertas

AlertasController · Presentación

▼

Casos de uso · Aplicación
ReportarMascotaPerdida, RegistrarAvistamiento, ActualizarEstadoReporte, ConsultarEstadoPublico

▼

Dominio y puertos
ReporteMascotaPerdida, RepositorioReportes, PublicadorEventos

▼

Adaptadores · Infraestructura
PostGIS y Outbox PostgreSQL

C4 · Caso de uso (pseudocódigo)

```
class ReportarMascotaPerdida {
  ejecutar(datos) {
    crea el reporte (dominio)
    lo guarda (puerto)
registra evento en Outbox
    devuelve id y estado
  }
}
```

El ejemplo ejecutable contiene 9 pruebas automáticas aprobadas.

Fuente: Brown (s. f.), The C4 model. Código completo en la carpeta tecnologia del repositorio.

Una arquitectura pensada para crecer



Alertas sin esperas

Quien reporta recibe respuesta al instante; el aviso masivo continúa aparte.



Crece por partes

La API se escala como unidad; los módulos mantienen límites para evolucionar sin acoplamiento.



Seguro por diseño

Caddy termina HTTPS; la API valida JWT y autorización por recurso.



Cambios sin sustos

Cambiar de proveedor o de base de datos no altera las reglas del negocio.

Las decisiones vigentes están registradas en ADR-001 a ADR-011.

Fuente: Nygard (2011).

Tecnologías abiertas, probadas y gratuitas

PWA
React, instalable en el móvil

Caddy + API
Caddy HTTPS + JWT en API

Monolito modular
Node.js / Express

Outbox
Eventos durables y caché cache-aside

PostgreSQL
Con PostGIS: consultas geoespaciales

Medios
Cloudflare R2 · medios externos

Despliegue
Docker Compose y GitHub Actions

Oracle Cloud
Free Tier: 2 OCPU y 12 GB totales (según disponibilidad)

Un solo lenguaje, JavaScript, de la pantalla al servidor.

Fuente: propuesta de arquitectura del proyecto.

Listo para empezar y para crecer

2 VM

1 OCPU y 6 GB cada una; sujetos a disponibilidad y límites del plan.

1000

sesiones simultáneas, perfil normal de carga; objetivo por validar.



Mañana

Perfiles abiertos para integrar veterinarias, tiendas y refugios.

Fuera de esta etapa: apps nativas, recomendaciones con aprendizaje automático y monetización.

Cómo crecer: optimizar consultas y caché; si las métricas lo justifican, replicar la API completa detrás de Caddy sin saturar PostgreSQL.

Fuentes

Bass, L., Clements, P., & Kazman, R. (2021). *Software architecture in practice (4.ª ed.). Addison-Wesley.*

Brown, S. (s. f.). *The C4 model for visualising software architecture. https://c4model.com*

Instituto Nacional de Estadística e Informática. (2025). *Tenencia y crianza de mascotas 2025 [Informe técnico]. ENAHO.*

Martin, R. C. (2017). *Clean architecture: A craftsman's guide to software structure and design. Prentice Hall.*

Newman, S. (2021). *Building microservices (2.ª ed.). O'Reilly Media.*

Nygard, M. (2011, 15 de noviembre). *Documenting architecture decisions. Cognitect. https://www.cognitect.com/blog/2011/11/15/documenting-architecture-decisions*

Ilustraciones, diagramas e iconos: elaboración propia.

ÚNETE A LA COMUNIDAD

Una comunidad que se habla, se cuida y se encuentra.



Siguiente paso: un piloto con dueños de mascotas, veterinarias y refugios de Ayacucho.

Gracias · RimayPet